Programming languages Java

Overview

Kitlei Róbert

ELTE
IK

What is included in the introductory section?

- Most of the topics mentioned during the semester
- We focus on the essence of the concepts
- For now we omit
  - ◇ problematic cases
  - ◇ deeper details
- We will not cover all the tools

History

- 1991–1996: design phase
- 2004: Java 5.0 (or 1.5 or Java 5)
  - ◇ from then on it resembles what appears on the slides
- SE LTS releases: Java 8, 11, 17, 21, 25, ...
  - ◇ Standard Edition, Long-Term Support
  - ◇ Java 25 is required to run the code on the slides!
- Almost full backward compatibility

API documentation: click here

ELTE
IK

jshell REPL environment

- REPL: read-evaluate-print loop
- Most Java statements end with ; here it may be omitted

```
> jshell
jshell> 1
$1 ==> 1
jshell> $1+2
$2 ==> 3
jshell> "Helló világ!"
$3 ==> "Helló világ!"
jshell> $1+$3
$4 ==> "1Helló világ!"
jshell> $3 + " □"
$5 ==> "Helló világ! □"
```

- If the □ symbol does not appear, run `chcp 65001` in the `cmd` terminal (UTF-8 mode) to fix it

Basic Java program

```
void main() {
    IO.println("Helló világ! 🌍");
}
```

This is a simplified form: “compact source file”

- Recommended only for simple, “single file” programs
- Java is case-sensitive: uppercase and lowercase letters differ!
- Longer form:

```
static void main() {
    IO.println("Helló világ! 🌍");
}
```

ELTE
IK

Basic Java program: trying it out

```
void main() {
    IO.println("Helló világ! 🌍");
}
```

Can be tried in the terminal and under `jshell`

- No output file is produced
- `jshell` will not be able to load more complex files this way

```
> java BasicHelloWorld.java
Helló világ! 🌍
> jshell
jshell> /open BasicHelloWorld.java
jshell> main()
Helló világ! 🌍
```

Basic Java program

Simplified form with command-line arguments

```
void main(String... args) {
    IO.println("First argument: " + args[0]);
    IO.println("Argument count: " + args.length);
}
```

Compile and run in one step

```
> java ParamsHelloWorld.java a b c 123
First argument: a
Argument count: 4
> jshell
jshell> /open ParamsHelloWorld.java
jshell> main("a", "b", "c", "123")
First argument: a
Argument count: 4
```

ELTE IK

Convenience tip #1: start with a clean slate

Empty the terminal window between two builds

- New outputs do not mix with previous ones
- Compile error messages start at the top of the window
  - ◇ Always read them!
  - ◇ The most important one is usually the first

Windows, PowerShell command prompt

```
cls && java BasicHelloWorld.java
```

Windows, `cmd` command prompt

```
cls ; java BasicHelloWorld.java
```

Linux/Mac, in any shell

```
clear && java BasicHelloWorld.java
```

ELTE
IK

Convenience tip #2: compile error messages

Many error messages can appear during compilation

- This is how to limit their number

```
cls && java -Xmaxerrs 3 Wrong.java
```

Convenience tip #3: runtime error messages

If the program compiles but is faulty, too many details may appear

- This is how to limit their number

```
import check.Use;

void main() throws Exception {
    Use.nicerLogs(); // magic!
    Integer.parseInt("not a number at all");
}
```

```
java -cp ".;checkthat6.jar" Wrong.java
```

ELTE
IK

Convenience tip #3: runtime error messages, output

Without the magic:

```
Exception in thread "main" java.lang.NumberFormatException:
                    For input string: "not a number at all"
    at java.base/java.lang.NumberFormatException.forInputString(
                                        NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:564)
    at java.base/java.lang.Integer.parseInt(Integer.java:661)
    at NicerLogs.wrong(NicerLogs.java:18)
    at NicerLogs.main(NicerLogs.java:9)
```

With the magic:

```
java.lang.NumberFormatException:
                    For input string: "not a number at all"
    at NicerLogs.wrong(NicerLogs.java:18)
    at NicerLogs.main(NicerLogs.java:9)
```

The slide-deck code examples

The code from the slides can be downloaded from Teams: `getting-started.zip`

- This contains additional items, e.g. common mistakes and how to avoid them

Full program form

```
// compact source files may not have packages
package hu.elte.inf.java.programming;

// every important part of the "standard toolset"
// automatically part of compact source files
import module java.base;

public class HelloWorld {
    public static void main(String... args) {
        System.out.println("Helló világ! " + args[0]);
    }
}
```

Place it as: `hu/elte/inf/java/programming/HelloWorld.java`

- The file name and its path are both fixed!

Full program form (2)

Compile and run

- run from the directory that contains the hu directory
- when compiling we give the *file* name, when running we give the *class* name

```
javac hu/elte/inf/java/programming/HelloWorld.java
        # output file: hu/elte/inf/java/programming/Hello.class
java hu.elte.inf.java.programming.Hello
```

- Can be combined into a single command so you only need to type it once

```
cls && javac hu/elte/inf/java/programming/HelloWorld.java
    && java hu.elte.inf.java.programming.Hello
```

Full program: REPL

```
> jshell
jshell> /open HelloWorld.java
jshell> HelloWorld.main("□")
Helló világ! □
```

Using code from another package

```
package utils.printing;

public class Class1 {
    // pure function: uses its args to compute its result
    public static String doubleString(String txt) {
        return txt + txt;
    }
}
```

Only classes from a named package can be imported; classes in the unnamed package cannot!

```
// if no package is specified, it goes into the "unnamed package"
// if no class is specified, the file name provides it

import utils.printing.Class1;     // a specific class
```

ELTE
IK

Using code from another package in a different fashion

```
package utils.printing;

import module java.base;

public class Class2 {
    // method with side effect: impure (reads and writes)
    public static void inputAndPrint(String prompt) {
        String txt = readln(prompt);
        IO.println(txt + txt);
    }
}

import utils.printing.*; // package utils.printing's every class

void main() {
    Class2.inputAndPrint("Enter the text to be doubled: ");
}
```

ELTE
IK

Enumeration type

```
public enum Rank { ACE, DEUCE, THREE, FOUR, FIVE, SIX, SEVEN, EIGH
public enum Suit { SPADES, CLUBS, HEARTS, DIAMONDS }
public enum GirlsBestFriend { DIAMONDS }

public record Card(Rank rank, Suit suit) {}

void main() {
    var rank = Rank.ACE;
    var suit = Suit.SPADES;

    IO.println(rank);
    IO.println(suit);
    IO.println(new Card(rank, suit));
    IO.println(new Card(Rank.ACE, Suit.SPADES));
    // Card[rank=ACE, suit=SPADES]
```

Enumeration type: további műveletek

```
void main() {
    printAllCards();
    readCard();
}

void printAllCards() {
    for (var rank : Rank.values()) {     // returns an (iterable)
        for (var suit : Suit.values()) { // every enum has it
            IO.println(rank + " of " + suit);
        }
    }
}

void readCard() {
    var rank = Rank.valueOf(readln("Enter the name of a rank:"));
    var suit = Suit.valueOf(readln("Enter the name of a suit: "));
```

Rekord (rendezett n-es; tuple)

There is no mutable data (state)

- Advantage: cannot be corrupted

```
public record Person(String name, int age) {}

void main() {
    Person jack = new Person("Jack", 12);
    var    jill = new Person("Jill", 34);

    IO.println(jack.name());
    IO.println(jill.age());

    IO.println(jack); // Person[name=Jack, age=12]
    IO.println(jill); // Person[name=Jill, age=34]
}
```

Deconstructing records

```
public record P(String name, int age) {}  // Person
public record Duo(Person p1, Person p2, int score) {}
void main() {
  var jack = new Person("Jack", 12);
  var jill = new Person("Jill", 34);
  var duo  = new Duo(jack, jill, 20);
  int result =
    switch (duo) {
    case Duo(P(var name1, var age1), var p2, var _) when age1   >
    case Duo(P p1, P p2, int _)                     when p1.age >
    case Duo(P(String name, int age), P _, int _)   when age    >
    case Duo(P p1, P p2, int score)
    default
    };
  print(result);
}
```

List

Generic type: takes a type parameter

```
void main() {
    var jack = new Person("Jack", 12);
    var jill = new Person("Jill", 34);

    List<Person> persons = List.of(jack, jill);

    IO.println(persons);
    IO.println(persons.size());
    IO.println(persons.get(0));

    for (int i = 0; i < persons.size(); ++i) {
        IO.println(persons.get(i));
    }
}
```

List otherwise

```
void main() {
    var persons = List.of(
        new Person("Jack", 12),
        new Person("Jill", 34)
    );

    IO.println(persons);
    IO.println(persons.size());
    IO.println(persons.get(0));

    // foreach loop
    for (Person person : persons) {
        IO.println(person);
    }
}
```

List of integers

Limitation: type arguments may not be primitives

- `char` → `Character`, `int` → `Integer`
- others: in uppercase

```
void main() {
    var             nums  = List.of(1, 2, 3);
    List<Integer> nums2 = List.of(1, 2, 3); // "wrapper type"

    // List<int> numsErr = List.of(1, 2, 3); // compile error
    List numsBad = List.of(1, 2, 3); // compiles but not very good

    IO.println(nums);
    IO.println(nums2);
}
```

Other important data structures

Type	Repeats	Keeps order	Insert	Search	Delete
`List`					
`Set`					
`Map` $\rightarrow$	:				

List: ArrayList

```
void main() {
    ArrayList<Card> deck  = new ArrayList<Card>();
    ArrayList<Card> deck2 = new ArrayList<>(); // shortcut, OK
    var deck3 = new ArrayList<Card>(); // type inference, OK
    var deck4 = new ArrayList<>();     // compiles but differs

    for (var rank : Rank.values()) {
        for (var suit : Suit.values()) {
            deck.add(new Card(rank, suit));
        }
    }

    IO.println(deck.contains(new Card(Rank.ACE, Suit.SPADES)));
    IO.println(deck);
}
```

ELTE
IK

List: other possibilities

```
void main() {
    List<Card> deck5 = new ArrayList<Card>();
    List<Card> deck6 = new ArrayList<>();

    List<Card> deck5 = new LinkedList<Card>();
    List<Card> deck6 = new LinkedList<>();

    List<Card> deckWrong = new List<>(); // !

    // ...
}
```

Arrays

```
Card[] reshuffleCards(Rank[] rs, Suit[] suits, int[] idxs) {
    if (rs.length!=suits.length || rs.length!=idxs.length) {
        throw new IllegalArgumentException("Size mismatch");
    }

    Card[] cards = new Card[rs.length];
    for (int i = 0; i < rs.length; ++i) {
        var idx = idxs[i];
        cards[idx] = new Card(rs[idx], suits[idx]);
    }
    return cards;
}
```

Using arrays

```
void main() {
    Rank[] ranks  = new Rank[] { Rank.JACK,   Rank.QUEEN  };
    var    ranks2 = new Rank[] { Rank.JACK,   Rank.QUEEN  };
    // shortcut, only in declarations
    Suit[] suits  =            { Suit.SPADES, Suit.HEARTS };
    int[]  idxs   =            { 1, 0 };

    int[] wrong = { 1, 0 }; // compile error

    Card[] result = reshuffleCards(ranks, suits, idxs);
    IO.println(result);
    // [LCardArray$Card;@3e92efc3
}
```

Arrays: advantages vs limitations

Arrays: benefits

- Fast operations
- Memory use is small
- Fixed element count at init

Drawbacks

- Fixed element count at init
- Basic operations (`toString`, `equals`, `hashCode`) function improperly
  - ◇ Not suitable as elements in data structures

What to do?

- Use `ArrayList`: similar efficiency, no drawbacks
- Use helper methods (`java.util.Arrays`)

```
void main() {
    // ...
    Card[] result = reshuffleCards(ranks, suits, idxs);
    IO.println(Arrays.toString(result));
    // [Card[rank=JACK, suit=SPADES], Card[rank=QUEEN, suit=HEARTS
```

OOP

Class: the structure of similar data

Object: individual filled with data, follows the structure of the class

```
public class Person {
    // state: fields (members)
    private String name;
    private int age;
    // initialization: constructors
    public Person(String name, int age) { ... }
    // operations: methods
    public void   ageUp()   { ... }
    public int    getAge()  { ... }
    public String getName() { ... }
}
```

OOP: usage

```
void main() {
    new Person("John", 99);          // instance

    var p = new Person("Jane", 66); // hard to use with no variabl
    IO.println(p.getAge());
    IO.println(p.ageUp());
    IO.println(p.getAge());
}
```

OOP

Initialization: the instance is created

- Fill it up with data
- Make sure that the data is valid

```
public class Person {
    // state: fields (members)
    private String name;
    private int age;
    // initialization: constructors
    public Person(String name, int age) {
        this.name = name;
        this.age  = age;
    }
    // operations: methods
    public void   ageUp()   { ++this.age; }
    public int    getAge()  { return this.age; }
```

OOP

this: a reference to the object of the call, always implicit

```
public class Person {
    private String name;
    private Person friend;

    public Person(/* Person this, */ String name, int age) {
        // accessing hidden name using this
        this.name = name;
        this.age  = age;
    }
}

// calls the above code
// during the call, "this" refers to the one being created
new Person("Bill",  33);
// outside of an instance, "this" has no meaning
```

OOP

this: a reference to the object of the call, always implicit

```
public class Person {
    ...

    public String getName(/* Person this */)   { return this.name;
    public Person getFriend(/* Person this */) { return this.frien
    public String makeFriend(/* Person this, */ Person other) {
        this.friend  = other;
        other.friend = this;
    }
}

Person p1 = new Person("Bill",  33);
Person p2 = new Person("Robin", 27);
p1.makeFriend(p2);
IO.println(p1.getFriend().getFriend().getName());
```

Indicating invalid operation: exception

```
public class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        if (name.equals("") || age < 0)
            throw new IllegalArgumentException(
                "Wrong name: " + name + " or age: " + age);
        this.name = name;
        this.age  = age;
    }
}
```

```
if (name.equals(""))     throw ...  // OK
if ("".equals(name))     throw ...  // OK
if (name.size() == 0)    throw ...  // OK
if (name == "")          throw ...  // wrong! In Java (almost) alwa
```

Indicating invalid operation: exception

```
void main() {
    try {
        new Person("Mr. Right",  29);
        new Person("Ms. Left",  -29);
        IO.println("Unreachable statement");
    } catch (IllegalArgumentException e) {
        IO.println(e.getMessage()); // only the message

        e.printStackTrace();        // detailed output
    }
}
```

Typical constructors

```
public class Person {
    private String name;
    private int age;
    // no-arg: makes a "central" object with dedicated content
    // public Person() {
    //     // don't create it if not applicable
    // }
    public Person(String name, int age) {
        if (/* the data in the args are invalid */)
            throw new IllegalArgumentException("...");
        this.fieldName = argName; // for each argument
    }
    // copy constructor
    public Person(Person other) {
        this.fieldName = other.fieldName; // for each field
    }
```

Typical operations: toString

```
public class Person {
    // getting the textual representation (not printing!)
    @Override public String toString() {
        // Java 15+
        return "%s (%d)".formatted(name, age);
        // Java 5+
        return String.format("%s (%d)", name, age);
        // with efficient text concatenation
        var sb = new StringBuilder();
        sb.append(name);
        sb.append(" (");
        sb.append(age);
        sb.append(")");
        return sb.toString();
        // using simple concatenation
        return name + " (" + age + ")";
```

Typical operations: equals and hashCode

Simple equality check: checks that all fields have the matching content

```
public class Person {
    @Override public boolean equals(Object other) {
        if (other == this)                    return true;
        if (!(other instanceof Person p)) return false;

        return Objects.equals(this.name, p.name) &&
               this.age == p.age;
    }
    // important companion method
    @Override public int hashCode() {
        return Objects.hash(this.name, this.age);
    }
}
```

Typical operations: equals and hashCode, usage

```
void main() {
    var p1 = new Person("Jack", 12);
    var p2 = new Person("Jill", 34);
    var p3 = new Person("Jack", 12);
    var p4 = p1;

    IO.println(p1 == p1);         // true : the same object
    IO.println(p1 == p3);         // false: different objects
    IO.println(p1 == p4);         // true : the same object, aliased
    IO.println(p1.equals(p1));    // true : with the shortcut
    IO.println(p1.equals(p2));    // false: different contents
    IO.println(p1.equals(p3));    // true : equivalent contents
}
```

ELTE
IK

Encapsulation

Goal: protect the innocent, uphold the law

```
public class Game {
    public  /*!!!*/ List<Player> players;
    public Game(List<Player> players) {
        this.players = players;
    }
    public List<Player> getPlayers() { return this.players; }
}

void main() {
    var g = new Game(pros); // accessed g's supposedly hidden stat
    IO.println(g.players);  // modified g's state without it knowi
    pros.remove(p1);        // again!
    g.getPlayers().add(p3);
}
```

Lifetime

```
void main() {
    new Player("Unused", 0); // eventually garbage collected
    var player = makePlayer();
    play(player);
    player = makePlayer(); // can't access the old object from now
                           // maybe not the end of the lifetime he
    // ...
}
Player makePlayer() {
    var name = "Eve";
    return new Player(name, 55);
}
void play(Player player) {
    IO.println(player.name + "(" + player.age + ") plays");
}
```

Aliasing

```
void main() {
    var player  = new Player("Gabe", 62);
    var player2 = player;
    player2.increaseAge();
    IO.println(player.getAge());              // 63
}
```

Empty reference

```
Point p = null;
p = new Point(4,6);
if (p != null) {
    p = null;
}
p.x = 3;    // NullPointerException
```

Empty reference

```
Point p = null;
p = new Point(4,6);
if (p != null) {
    p = null;
}
p.x = 3;    // NullPointerException
```

I call it my billion-dollar mistake.
— Tony Hoare, creator of **null**

Empty reference

```
Point p = null;
p = new Point(4,6);
if (p != null) {
    p = null;
}
p.x = 3;    // NullPointerException
```

I call it my billion-dollar mistake.
— Tony Hoare, creator of **null**

The dark side of the Force are they. Easily they flow, quick to join you in a fight. If once you start down the dark path, forever will it dominate your destiny, consume you it will... — Yoda

Empty reference

```
Point p = null;
p = new Point(4,6);
if (p != null) {
    p = null;
}
p.x = 3;    // NullPointerExcept
```

I call it my billion-dollar mistake.
— Tony Hoare, creator of **null**

The dark side of the Force are they. Easily they flow, quick to join you in a fight. If once you start down the dark path, forever will it dominate your destiny, consume you it will... — Yoda

Inheritance between classes

Class is derived from another class

- technically: fields, methods get inherited
  - ◇ (but not the constructors)
  - ◇ avoids code duplication

```
public class Insect { ... }                // more general class
public class Bug extends Insect { ... } // more special class:
                                           //   bugs ⊆ insects
```

- conceptionally: creates a subtype
  - ◇ Liskov substitution principle (LSP)

```
Insect insect = new Bug();    // OK: all bugs are insects
Bug    bug    = new Insect(); // this is a bug in the program
```

Class hierarchy

Inheritance relations form a tree hierarchy

- upwards: parent class, base/ancestor class
- downwards: child class, descendant/subclass

```
public class Being { ... }
public class Human   extends Being { ... } // humans⊆beings
public class Student extends Human { ... } // students⊆humans⊆bein
public class Teacher extends Human { ... } // teachers⊆humans⊆bein
```

At the top: `java.lang.Object`

- becomes the parent when left unspecified
- the `java.lang` package is autoimported

```
public class Insect extends java.lang.Object { ... }
public class Being  extends Object           { ... }
```

Calling the parent constructor

```
class Being {
    private int legCount;
    public Being(int lC) { this.legCount = lC; }
}
class Insect extends Being {
    public Insect(int legCount) {
        if (legCount != 6)   throw ...;
        super(legCount); // must call parent constructor
    }
}
class Bug extends Insect { public Bug() { super(6); } }
void main() {
    Being  longestMillipede = new Being(1306);
    Insect silverBeatles    = new Insect(8);
    Bug    buggy            = new Bug();
}
```

Overriding: changing the definition

```
class Being {
    private int legCount;
    public Being(int lC) { this.legCount = lC; }
}
class Insect extends Being {
    public Insect(int legCount) {
        if (legCount != 6)   throw ...;
        super(legCount); // must call parent constructor
    }
}
class Bug extends Insect { public Bug() { super(6); } }
void main() {
    Being  longestMillipede = new Being(1306);
    Insect silverBeatles    = new Insect(8);
    Bug    buggy            = new Bug();
}
```

Dynamic binding: method calls vs hierarchy

```
class Being {
    public abstract void eat(); // cannot implement yet
}
class Insect extends Being { ... }
class Bug extends Insect {
    @Override
    public void eat() { suck(); }
}
class Human extends Being {
    @Override
    public void eat() { eatWith("fork", "knife"); }
}
class Patron extends Human {
    @Override
    public void eat() { super.eat(); pay(); }
}
```

Interface inheritance

Objects have two aspects

- hidden: its representation (state; fields)
- public interface: its methods

Separate type for it: **`interface`**

- abstraction: "what?" independent of "how?"
  - ◇ interfaces list not-yet-fulfilled promises; contracts

Interface inheritance: example

```
public interface Payment {
    void pay(Human recipient, int sum);
}
public class CreditCard implements Payment {
    private Bank   bank   = ...;
    private String fromId = "Air Shield";
    private int    pwd    = 12345;
    @Override public void pay(Human recipient, int sum) {
        bank.sendMoney(recipient, fromId, pwd, sum);
    }
}
public class Cash implements Payment {
    @Override public void pay(Human recipient, int sum) {
        // ...
    }
}
```